ConvPE Substantially Improves Transformers

Wang Zehao
Nanjing University
wzhagi@163.com

ABSTRACT

Convolutional Position Embedding (ConvPE) achieves substantial reductions in validation loss compared with the Rotary Position Embedding (RoPE) baseline, while adding only a small number of backbone parameters (+0.15% params, -3.83% loss; +1.87% params, -8.03% loss). As model scale increases, the best configuration achieves a 10.3% loss reduction with only a 4.34% increase in backbone parameters, demonstrating high performance and parameter efficiency. ConvPE uses convolution to generate attention score biases, making it compatible with various Transformer architectures and high-dimensional modalities. The code for ConvPE is publicly available at `https://github.com/wzhagi/convpe`.

Keywords Position Embedding · Language Models · Transformers · Attention

1 Introduction

Transformers[1] are insensitive to input order. However, word order is crucial for language understanding. Explicit or implicit position embeddings are therefore necessary to inject order information into the model[2,3,4].

This paper proposes ConvPE, a novel and efficient position embedding scheme. ConvPE embeds position information into attention score biases, trading a minor parameter overhead for significant performance gains, as shown in Figure 1. Its design allows the model to switch freely between different attention masks and is compatible with various Transformer architectures. It also naturally supports high-dimensional modalities with n-dimensional convolution.

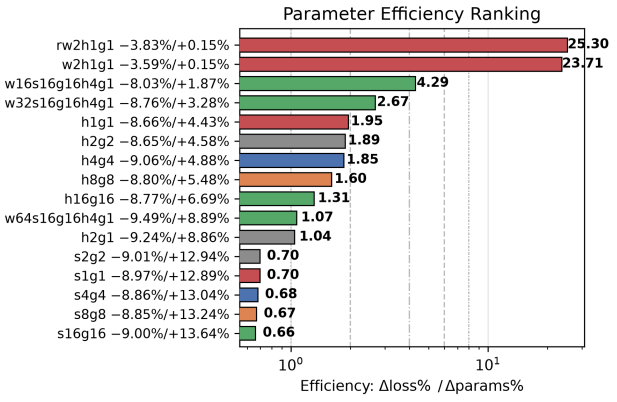


Figure 1: Parameter efficiency Top16 of experiments in section 4.1

The main contributions of this paper are as follows:

1. We propose the ConvPE architecture that combines high performance and broad compatibility.

2. We systematically study the influence of ConvPE architecture and hyperparameters through language model training.

3. We verify the effectiveness of ConvPE in scaling experiments.

4. We verify the effectiveness of ConvPE2D in visual model training.

2 Background

2.1 Attention Mechanism

Given the input query $Q \in \mathbb{R}^{T \times d_k}$, key $K \in \mathbb{R}^{S \times d_k}$, and value $V \in \mathbb{R}^{S \times d_v}$, the computation of scaled dot-product attention[1] $O \in \mathbb{R}^{T \times d_v}$ is:

$$O = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \mathbf{B}\right)V$$

T is the query sequence length. S is the key/value sequence length. $\mathbf{B} \in \mathbb{R}^{T \times S}$ is the attention score bias. Causal models introduce a causal mask $M \in \mathbb{R}^{T \times S}$ that sets the attention score bias $\mathbf{B}$ for invisible positions to $-\infty$. With batch size and multi-head settings, Q, K, V, O additionally have B, H dimensions.

2.2 Position Embedding

Existing position embedding schemes can be roughly divided into five categories:

1. **Absolute Position Embedding**[1]: adds a fixed positional bias, usually sine waves, to the input vector. It is simple to implement, but its performance is usually limited.

2. **Learnable Positional Bias**[2,3]: adds a learnable positional bias to the input sequence. It is typically adopted in limited-window settings, such as spatial dimensions in vision models or fixed-rate compression.

3. **Rotary Position Embedding**[4]: encodes position as a rotation matrix and applies it to queries and keys, so that attention scores depend on relative positions. RoPE has been widely adopted in modern large language models, enabling long-context extension via frequency scaling, such as YaRN[5].

4. **Attention Score Bias**: adds a positional bias B to attention scores, such as T5 relative position bias[6] and ALiBi[7]. RoPE can be viewed as a rotation-based equivalent, as it also makes attention scores depend on relative positions.

5. **No Position Embedding (NoPE)**[8]: adds no explicit position embeddings; instead, it relies on model structure and causal attention masks to implicitly learn positional information, as in Kimi K3[9], requiring no explicit position extension.

3 Method

ConvPE is a convolutional attention score bias method. Its goal is to construct an attention score bias tensor $\mathbf{B_{ConvPE}} \in \mathbb{R}^{B \times H \times T \times W}$ for self-attention tokens within the convolution window. Tokens outside the window use no attention score bias.

$$\mathbf{B}^{b,h,t,s} = \begin{cases} \mathbf{B}^{b,h,t,w}_{\text{ConvPE}}, & w \in [0, W-1] \\ 0, & \text{otherwise.} \end{cases}$$

$$w = s - t + W - 1$$

The hyperparameters include:

- W: convolution window size.
- C_s: number of convolution input channels for shared projection.
- G_s: number of convolution groups for shared projection.
- C_h: number of convolution input channels for headwise projection.
- G_h: number of convolution groups for headwise projection.

There are two ways to obtain $\mathbf{B}$ from the layer input $X \in \mathbb{R}^{B \times T \times D}$. ConvPE adopts the form of adding the two biases:

$$\mathbf{B}_{ConvPE} = \mathbf{B}_s + \mathbf{B}_h$$

3.1 Shared

The shared projection layer W_s is used to downsample X. To adapt to the convolution layer input format (B, C, T), the last two dimensions need to be swapped:

$$I_s = (W_s X)^\top$$

After left-padding $W - 1$ zeros, pass it through a convolution layer:

$$O_s = \mathrm{Conv1d}(C_s, H \times W, W, G_s)(\mathrm{Pad}(I_s))$$

$\mathbf{B}_s \in \mathbb{R}^{B \times H \times T \times W}$ is obtained by applying dimension transformation to O_s.

Note that unflatten $O_s \in \mathbb{R}^{B \times C \times T}$ into $O_s \in \mathbb{R}^{B \times (W \times H) \times T}$ or $O_s \in \mathbb{R}^{B \times (H \times W) \times T}$ is asymmetric when $G_s \neq 1$.

The projection parameter count is $D \times C_s$, and the convolution parameter count is $W^2 \times H \times C_s / G_s$.

3.2 Headwise

The Headwise projection layer W_h is used to downsample X:

$$I_h = W_h X$$

Then convert $I_h \in \mathbb{R}^{B \times T \times (H \times C_h)}$ into $I_h \in \mathbb{R}^{(B \times H) \times C_h \times T}$ using dimension transformations. After left-padding $W - 1$ zeros, pass it through a convolution layer:

$$O_h = \mathrm{Conv1d}(C_h, W, W, G_h)(\mathrm{Pad}(I_h))$$

$\mathbf{B}_h \in \mathbb{R}^{B \times H \times T \times W}$ is obtained by applying dimension transformation to $O_h \in \mathbb{R}^{(B \times H) \times W \times T}$.

The projection parameter count is $D \times H \times C_h$, and the convolution parameter count is $W^2 \times C_h / G_h$.

4 Experiments

All models used in these experiments are Multi-Query Attention (MQA)[10], with $K = V$[11]. The attention layers parameter count is $dim_model \times dim_heads \times (2 \times num_heads + 1)$. The feed-forward layers use SwiGLU[12] with a total parameter count of $dim_model^2 \times 8$. The models are trained for one epoch on WikiText-103[13], after which validation loss is recorded. We used the Hugging Face Trainer[14] for training, AdamW[15] as the optimizer, and GPT2Tokenizer[3] as the tokenizer. The training hyperparameters are: batch_size=4, learn_rate=3e-4, block_size=1024, lr_scheduler_type="cosine", weight_decay=0.01, max_grad_norm=1.0, seed=42.

4.1 Architecture and Hyperparameter Evaluation

The model parameters used in this round are $N_{layers} = 6$, $D_{model} = 192$, $N_{heads} = 3$, $D_{head} = 64$. The main model has 2.3M parameters, and the embedding layer has 9.6M parameters.

ConvPE has 5 hyperparameters, for example: $W = 64, C_s = 16, G_s = 8, C_h = 4, G_h = 1$. We abbreviate this configuration as the label $w64s16g8h4g1$. If RoPE is used, we prefix the label with r, for example $rw64s16g8h4g1$. Since most experiments use $w128$, this configuration is omitted from the labels. If shared or headwise parameters are absent from the label, the corresponding mode is not used.

4.1.1 Single-Mode Hyperparameter Evaluation

Figure 2 shows the training results under single-mode configurations. Under all configurations, performance is substantially improved over the RoPE baseline. Intuitively, as G increases, parameter efficiency increases and performance decreases; as C increases, performance first improves rapidly and then enters a noisy slow improvement phase. We observed that $C_h \in [1, 2, 4]$ has the highest performance and parameter efficiency. Although shared mode is less parameter-efficient, it has greater potential for parameter scaling. $C_s = G_s$ has the fewest parameters and therefore the highest parameter efficiency.

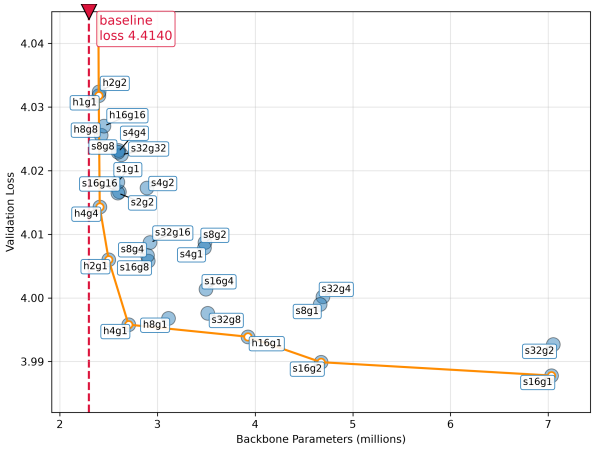


Figure 2: Single-Mode experiments validation loss

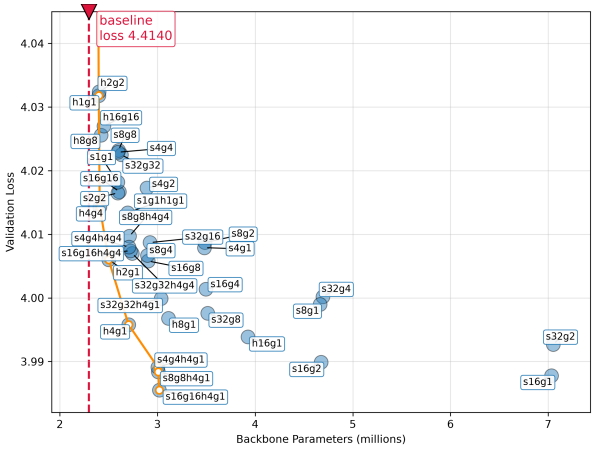


Figure 3: Hybrid-Mode experiments validation loss

4.1.2 Hybrid-Mode Hyperparameter Evaluation

The previous section shows:

- $C_s = G_s$ has relatively high parameter efficiency.
- $h1g1$ has the highest parameter efficiency.
- $h4g4$ has similar parameter efficiency but higher performance.
- $h4g1$ achieves performance comparable to $h16g1$ without a large increase in parameter count.

Therefore, this round mainly evaluates combinations of the above four configurations.

Figure 3 shows that $s1g1h1g1$, which has the highest parameter efficiency, does not surpass $h4g1$ in either performance or parameter efficiency. Thus, hybrid modes are used mainly to improve performance rather than parameter efficiency. We observe that $s4g4h4g1$, $s8g8h4g1$, and $s16g16h4g1$ achieve the best performance at almost the same parameter scale, with $s16g16h4g1$ achieving the best performance so far.

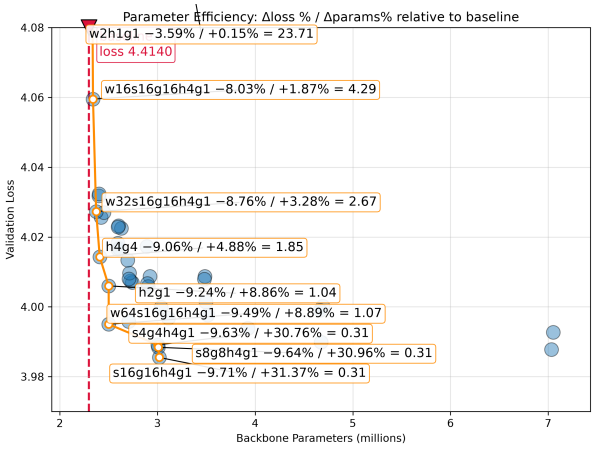


Figure 4: Window-Size experiments validation loss and parameter efficiency $\Delta loss\%/\Delta params\%$

config	RoPE val loss	NoPE val loss
baseline	**4.414**	4.6948
w2h1g1	**4.2448**	4.2554
s4g4h4g4	4.0195	**4.0080**
s16g16h4g1	4.0113	**3.9855**

Table 1: NoPE vs RoPE

4.1.3 Window-Size Evaluation

The parameter counts of both modes contain a W^2 term, indicating that reducing the window size can quickly lower the parameter count. Since $s16g16h4g1$ performed best in the previous round but has a relatively large parameter count, we evaluated $w64$, $w32$, and $w16$ for this configuration. Figure 4 shows that this series achieves the best performance at

each parameter scale. We then tried $w2h1g1$, which has the theoretically smallest parameter count. The results show that reducing window size does not make ConvPE perform worse than the RoPE baseline; with only a 0.15% increase in parameter count, the loss decreases by 3.659%, reaching a ratio of 23.71.

As the window size increases, performance continuously improves while high parameter efficiency is maintained. This appears to be a promising scaling dimension.

4.1.4 NoPE vs RoPE

Since the $w2h1g1$ window is very small, we tried combining it with RoPE to achieve higher performance. Table 1 shows that $rw2h1g1$ achieves the highest efficiency without increasing the parameter count, the loss decreases by 3.83% relative to the RoPE baseline, reaching a new highest efficiency ratio of 25.3. We also tested other configurations in RoPE mode, and all performed worse than NoPE mode.

4.2 Training on Larger Models

The configuration $s16g16h4g1$ has the best balance between performance and parameter efficiency on small models. Larger models may exhibit different hyperparameter behavior and require further experimental validation. The model parameters used in this round are $N_{layers} = 12$, $D_{model} = 768$, $N_{heads} = 12$, $D_{head} = 128$. The main model has 86M parameters, and the embedding layer has 38M parameters.

config	val loss	params	eff.
baseline	3.8055	86.1M	-
s16g16h4g1 C=WH	**3.4136**	+3.74M	**2.37**
s16g16h4g1 C=HW	3.4160	+3.74M	2.36
rs16g16h4g1 C=HW	3.4526	+3.74M	2.13

Table 2: Channel-splitting experiments results

config	val loss	params	eff.
s16g16h4g1	3.4136	**+3.74M**	**2.37**
s32g32h4g1	3.4185	+3.88M	2.26
s16g16h8g1	3.4169	+4.96M	1.77
s16g8h4g1	**3.4127**	+6.09M	1.46
s16g1h4g1	3.4253	+39.12M	0.22

Table 3: Hyperparameter search results

4.2.1 Channel-Splitting and RoPE Integration

Table 2 shows that as the backbone model scales, ConvPE performance improves. ConvPE + RoPE still performs worse than ConvPE alone. As the parameter proportion shrinks, parameter efficiency improves substantially, from 0.31 in the previous section to 2.37 (7.65×).

Note that unflattening $O_s \in \mathbb{R}^{B\times C\times T}$ into $O_s \in \mathbb{R}^{B\times (W\times H)\times T}$ or $O_s \in \mathbb{R}^{B\times (H\times W)\times T}$ is asymmetric when $G_s \neq 1$. Experiments show that $C = W \times H$ is slightly better, but all earlier experiments used $C = H \times W$. We did not rerun the early experiments because the improvement is so small. However, the following experiments start to use $C = W \times H$ instead.

4.2.2 Hyperparameter Search

We searched hyperparameters in the neighborhood of $s16g16h4g1$. Table 3 shows that $s16g16h4g1$ maintains high parameter efficiency and high performance.

4.2.3 No-Projection Configurations

sx means removing W_s and using X as the projection layer output.

hx means removing W_h and using X as the projection layer output.

Theoretically, any linear mapping $W \times D \to H \times W$ is equivalent to $sxg1$ while other configurations are its low-rank representations. The parameter count then becomes $W^2 \times H \times D$, which is too large and can only be tried with small windows or extremely large models.

Table 4 shows that both no-projection configurations perform worse. However, removing the shared projection layer can improve parameter efficiency.

config	val loss	params	eff.
s16g16h4g1	**3.4136**	+3.74M	2.37
sxg768h4g1	3.4211	**+3.59M**	**2.42**
s16g16hxg64	3.4545	+4.87M	1.63

Table 4: Experimental results for no-projection configurations

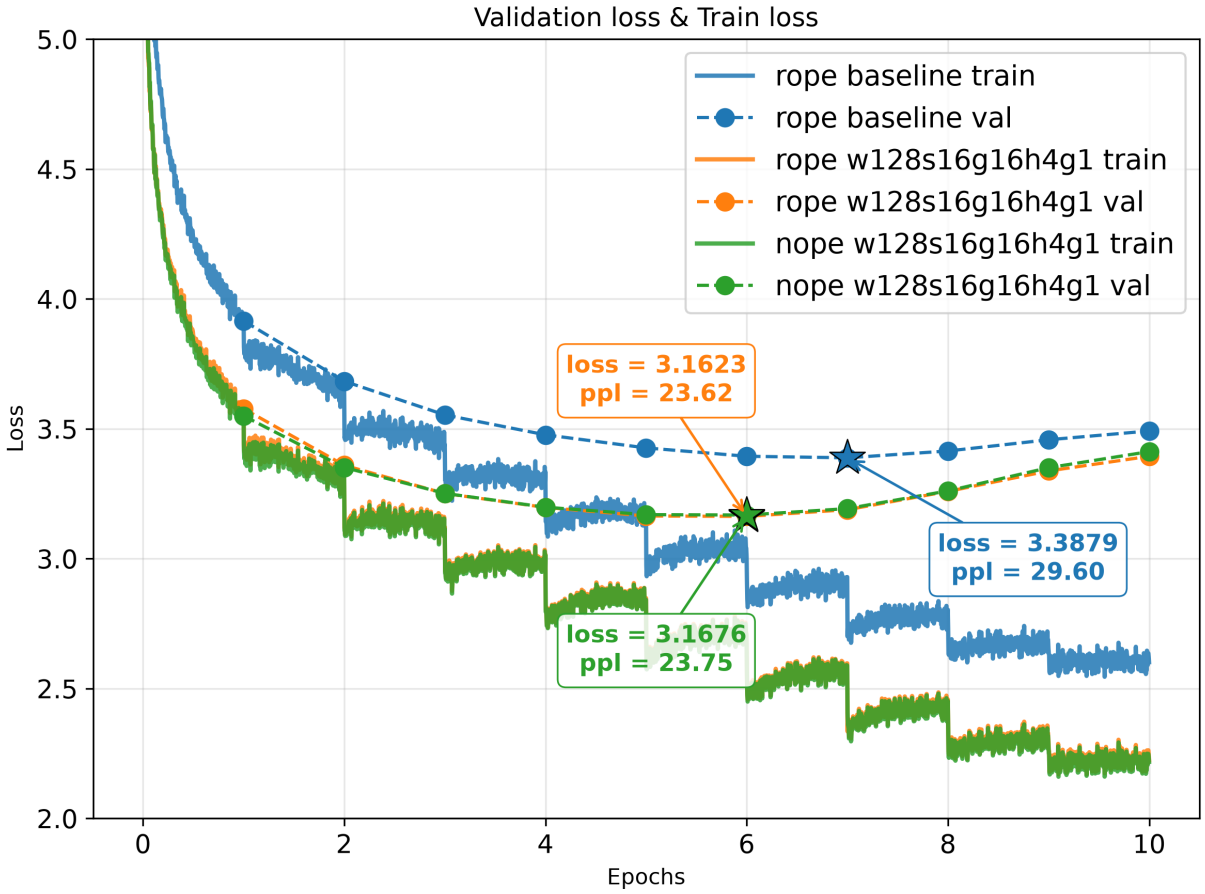


Figure 5: Training dynamics on WikiText-103 for 10 epochs

4.2.4 Multi-Epochs Training

We trained ConvPE models and a RoPE baseline from scratch on WikiText-103 for 10 epochs. Figure 5 demonstrates the remarkable superiority of the ConvPE architecture. The training and validation losses of ConvPE consistently remain substantially below those of the baseline, showing exceptional training efficiency and generalization capabilities. Notably, training ConvPE for merely one epoch surpasses the baseline trained for three epochs, and by epoch two, it already achieves a lower validation loss than the baseline's absolute best. Quantitatively, the best ConvPE validation loss reaches 3.1623 (ppl 23.62), which is 6.66% lower than the baseline's best of 3.3879 (ppl 29.60).

While ConvPE fundamentally outperforms the baseline, the performance gap between its RoPE and NoPE variants is marginal, though their generalization dynamics differ slightly. In low-epoch scenarios, the NoPE variant achieves a lower validation loss, indicating better early-stage generalization compared to its RoPE counterpart. Conversely, under multi-epoch training conditions, the RoPE variant catches up and yields a slightly lower generalization loss (e.g., 3.1623 vs. 3.1676 at epoch 6), and shows less overfitting with similar training loss. Additionally, regarding training set optimization, the NoPE variant maintains a consistently lower training loss than the RoPE variant throughout the entire training process.

config	params	CIFAR-10	CIFAR-100
baseline	5.3M	0.909	0.631
ConvPE2D w3	5.4M	**0.915**	0.641
ConvPE2D w5	5.5M	0.906	**0.653**

Table 5: Validation accuracy for 2D vision experiments on CIFAR-10 and CIFAR-100

4.3 2D Vision Position Embedding

To produce 2D positional embeddings, we use 2D convolutional layers. Instead of using a local windowed attention bias $\mathbf{B_{ConvPE}} \in \mathbb{R}^{B\times H\times T\times W}$, we use a global attention bias $\mathbf{B_{ConvPE2D}} \in \mathbb{R}^{B\times H\times T\times T}$, where T is the sequence length after embedding the image. Note that the input window size W and the output window size W_{out} in output channel dimension of the convolution layer can be different.

We trained Multi-Head Attention (MHA) ViT-Tiny[16] models with configuration $N_{layers} = 12$, $D_{model} = 192$, $N_{heads} = 3$, $D_{head} = 64$. ConvPE2D configuration is still $s16g16h4g1$ but with $W_{out} = T$. We used the Muon[17] optimizer to optimize the 2D parameters, excluding the embedding layer, with a learning rate of 3e-3, while the remaining parameters were optimized with AdamW at a learning rate of 3e-4. We trained the models on CIFAR-10 and CIFAR-100[18] for 100 epochs with batch_size=256, patch_shape=(4,4).

Table 5 shows that ConvPE transfers effectively to Vision Transformers and outperforms the RoPE baseline. Increasing the window size improves validation accuracy on the harder dataset CIFAR-100, while CIFAR-10 remains nearly saturated and favors the smaller w3 window.

5 Conclusions and Future Work

ConvPE delivers substantial performance gains at a minor parameter cost, providing a powerful and parameter-efficient alternative for Transformer position modeling.

Based on our experiments, we summarize the following practical tuning guidelines:

- **Adopt $s16g16h4g1$ as the default configuration.** It achieves an optimal balance between performance and parameter efficiency.
- **Scale the window size.** Increasing the window size yields continuous performance improvements.
- **Expect larger gains at larger scales.** As the backbone model grows, ConvPE tends to deliver greater performance improvements and higher parameter efficiency.

ConvPE is compatible with different attention masks and KV-cache concatenation, and it naturally supports multidimensional extension. Future work can explore the following directions:

- **Kernel fusion**. Develop high performance kernels for ConvPE.
- **Scaling behavior.** Investigate ConvPE with larger Transformers and datasets, and identify configuration patterns at scale.
- **Long-context modeling.** Evaluate ConvPE in long-context settings, where positional generalization is particularly important.
- **Architectural integration.** Combine ConvPE with Sliding Window Attention (SWA)[19], Compressed Sparse Attention (CSA)[11], and other Transformer variants.
- **Multidimensional modalities.** Extend ConvPE to high-dimensional modalities such as video, speech, and multimodal learning.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[4] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[5] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=wHBfxhZu1u`.

[6] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[7] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

[8] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. *arXiv preprint arXiv:2305.19466*, 2023.

[9] Kimi Team. Kimi k3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026. URL `https://arxiv.org/abs/2607.24653`.

[10] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[11] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL `https://arxiv.org/abs/2606.19348`.

[12] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[13] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[14] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[17] Keller Jordan. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[18] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[19] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.